Jacobian, Hessian & Multivariate Taylor

When derivatives become matrices

Prof. Nipun Batra

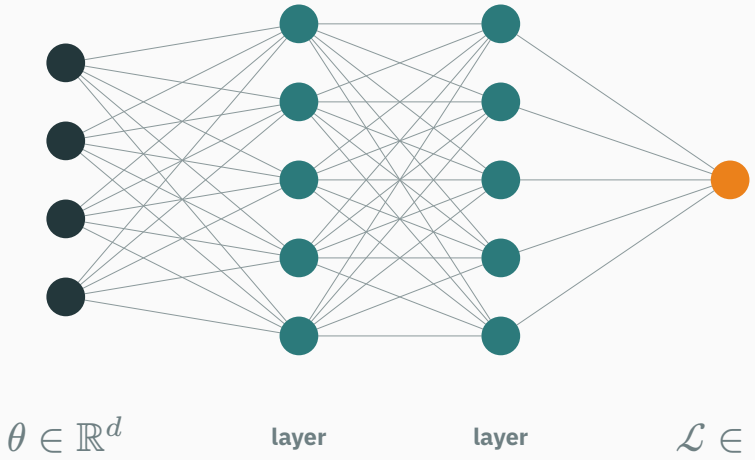
Mathematical Foundations for AI · IIT Gandhinagar

Derivatives of vector-valued functions and large losses

The function AI actually trains

V VISUAL

A neural network with its loss attached is **one function**: parameters in, a single number out.



For real models $d \approx 10^6$ to 10^{12}. Training (L17) needs this function's **derivative**.

But what single object is "the derivative" of $\mathbb{R}^{10^6} \to \mathbb{R}$? And between the layers sit maps $\mathbb{R}^n \to \mathbb{R}^m$ — vectors in, **vectors out**. L7's one number cannot be the answer.

Scope and notation for this lecture

The derivative grows up in two steps:

- **vector in, vector out** $\rightarrow$ the derivative is a matrix: the **Jacobian**;
- **curvature** of a surface $\rightarrow$ also a matrix: the **Hessian** — and its eigenvalue signs (L5!) say whether you're in a bowl, on a saddle, or in a trough.

This is the **heaviest-notation lecture of the course**, so we fix one working rule: every idea is developed on $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ first — small enough to check by hand. The general case is always the **same picture, more rows**.

By the end, "Jacobian" and "Hessian" will be **pictures**, not notation: a parallelogram, and a bowl.

The derivative table — today's map

V VISUAL

One table to navigate the whole lecture. Two rows you own; two we fill today:

function	derivative	shape	what it is
$f : \mathbb{R} \to \mathbb{R}$	$f'(x)$	a number	slope of the local line (L7)
$f : \mathbb{R}^n \to \mathbb{R}$	$\nabla f(\boldsymbol{x})$	vector, n	all partials; points straight uphill (L8)
$f : \mathbb{R}^n \to \mathbb{R}^m$	**?**	**?**	today
$f : \mathbb{R}^n \to \mathbb{R}$, order 2	**?**	**?**	today

Same idea every row: the derivative supplies the coefficients of the best local linear approximation; its container changes with the function's input and output shapes.

Learning outcomes

By the end of this lecture you will be able to:

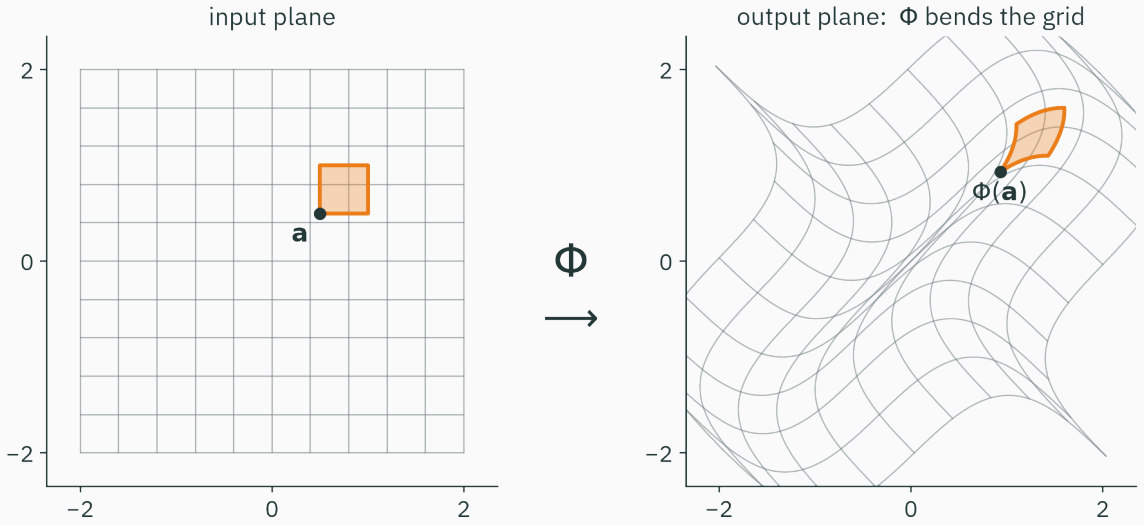
1. Compute a 2×2 **Jacobian**, read it as a local linear map, and read $|\det J|$ as area scaling.
2. Assemble a **Hessian** and explain entry (i, j) as "how slope i responds to nudge j".
3. Classify $x^\top Ax$ as bowl / saddle / trough from **eigenvalue signs**, with 2×2 hand tests.
4. Derive ⭐ $\nabla(x^\top Ax) = (A + A^\top)x$.
5. Write the **second-order Taylor expansion** and identify the terms used by first- and second-order optimization methods.
6. Keep shapes straight: gradient-as-column, J is $m \times n$, chain rule = matrix product.

Vector in, vector out: the Jacobian

A function that moves points

V VISUAL

$\Phi : \mathbb{R}^2 \to \mathbb{R}^2$ eats a point, returns a point. Feed it a whole grid and watch space bend:



Global map, local linear behaviour

$\Phi(x, y) = (x + 0.6 \sin 1.6y,\ y + 0.6 \sin 1.6x)$ is globally curved: it bends the full grid.

At sufficiently small scale, the orange square is only **leaned and stretched**. The Jacobian will be the matrix that describes that local move. Abstractly, any differentiable vector-valued layer has this input–output form.

The one case we already own: a linear map

For a **linear** map (L4: “a matrix is a verb”) the nudge formula is exact, everywhere, with no zooming:

$$F(\boldsymbol{x}) = A\boldsymbol{x} = \begin{pmatrix} 2 & 1 \\ 3 & -2 \end{pmatrix} \boldsymbol{x} \quad \Rightarrow \quad F(\boldsymbol{a} + \boldsymbol{\delta}) = F(\boldsymbol{a}) + A\boldsymbol{\delta}$$

Nudge the input by $\boldsymbol{\delta}$ — the output moves by $A\boldsymbol{\delta}$. **The matrix A is the derivative**, at every point.

Today’s target: for a curvy Φ, find the matrix that plays the role of A **near one point**.

Numbers: nudge the polar map, watch both outputs

Take the most useful curvy $\mathbb{R}^2 \to \mathbb{R}^2$ map there is — polar coordinates — at $(r, \theta) = \left(2, \frac{\pi}{3}\right)$:

$$\Phi(r, \theta) = \Big(\underbrace{r\cos\theta}_{x}, \underbrace{r\sin\theta}_{y} \Big), \qquad \Phi\left(2, \frac{\pi}{3}\right) = (1.000, 1.732)$$

Nudge each input by 0.01, one at a time (L7's nudge table, run twice per output):

nudge	**Δx, per unit nudge**	**Δy, per unit nudge**
$r : 2 \to 2.01$	$+0.00500/0.01 = 0.500$	$+0.00866/0.01 = 0.866$
$\theta : \frac{\pi}{3} \to \frac{\pi}{3} + 0.01$	$-0.01737/0.01 = -1.737$	$+0.00991/0.01 = 0.991$

Two inputs × two outputs = **four** sensitivities. One number was never going to be enough — but four numbers have an obvious home.

Pack the four sensitivities: the Jacobian

House rule: **one row per output, one column per input.** Each entry is a partial derivative (L8):

$$J = \begin{pmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{pmatrix} = \begin{pmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{pmatrix} \underset{(r,\theta)=(2,\pi/3)}{\Rightarrow} \begin{pmatrix} 0.500 & -1.732 \\ 0.866 & 1.000 \end{pmatrix}$$

Exactly the four measured ratios (the -1.737 and 0.991 were drifting here as the nudge shrank).

- Row 1 = $\nabla x^\top$: how output x feels each input — a gradient lying on its side.
- Sanity check on the linear map: every partial of Ax is a constant, and $J = A$. ✓

The picture: little squares become parallelograms

V VISUAL

True image of a tiny square (ink) vs the parallelogram J makes of it (orange) — the mismatch **dies faster than ε**:

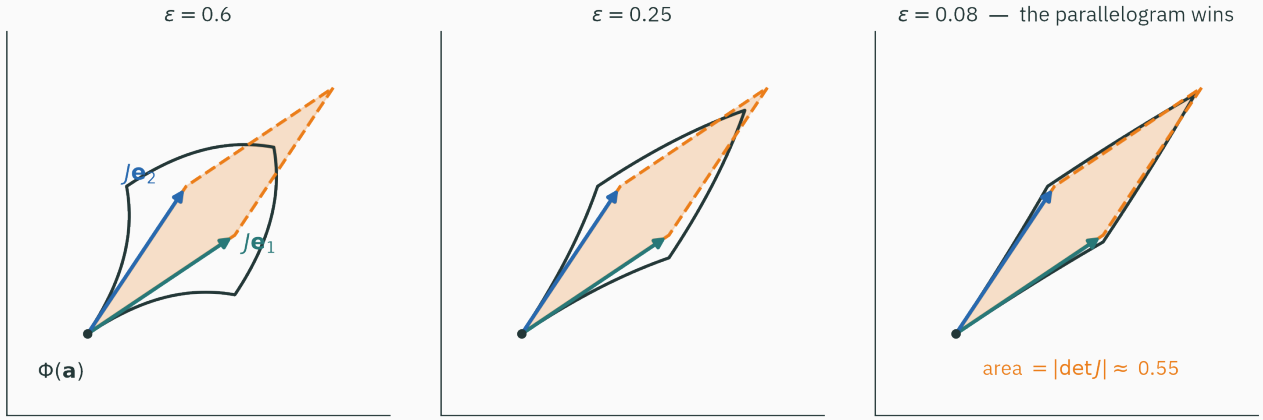


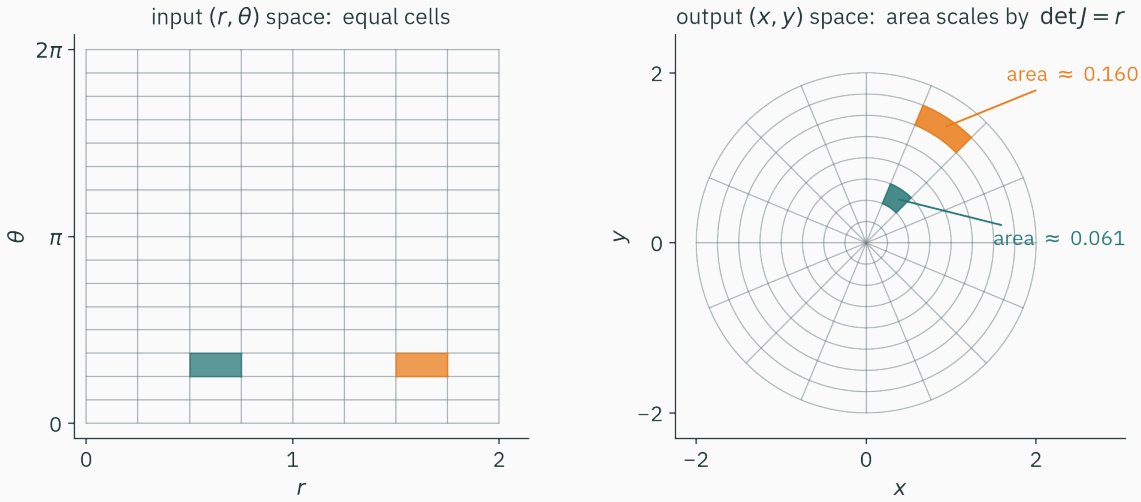
image of the ε-square at **a** (ink), recentered at $\Phi(\mathbf{a})$ and rescaled by $1/\varepsilon$ vs the parallelogram with edges $J\mathbf{e}_1, J\mathbf{e}_2$ (orange)

$$\Phi(a+\delta) \approx \Phi(a) + J(a)\,\delta$$

The Jacobian is the linear map that matches Φ to first order near a.

$|\det J|$ is the local area-scaling factor

L4: a determinant is the area-scaling factor of a linear map, so $|\det J|$ is the local area-scaling factor of Φ:



For polar: $\det J = r\cos^2\theta + r\sin^2\theta = r$ — equal input cells land with areas 0.061 and 0.160: a $2.6\times$ ratio, **exactly the ratio of their centre radii**. At $r = 0$: $\det J = 0$, the whole θ-axis is **crushed to a point** (L4′s null space, gone local).

Code: PyTorch measures the same matrix

```
import torch
from torch.autograd.functional import jacobian

def polar(p):
    r, th = p
    return torch.stack([r * torch.cos(th), r * torch.sin(th)])

J = jacobian(polar, torch.tensor([2.0, torch.pi / 3]))
# tensor([[ 0.5000, -1.7321],
#         [ 0.8660,  1.0000]])
torch.det(J)          # tensor(2.0000)  = r  ✓
```

The four measured ratios agree to four decimals with the derived factor $r = 2$.

L12 uses this factor in change of variables: probability density is divided by the local volume-scaling factor $|\det J|$.

The general case: same picture, more rows

For $f : \mathbb{R}^n \to \mathbb{R}^m$ — nothing new, just a bigger crate (m outputs stacked, n inputs across):

$$J(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{m \times n}, \qquad J_{ij} = \text{how output } i \text{ feels input } j$$

The whole derivative table collapses out of this one object:

- $m = 1$: a single row — the gradient lying on its side, $J = \nabla f^\top$ (L8).
- $m = n = 1$: a 1×1 matrix — L7′s lone number $f'(x)$.

There is only one derivative idea. The Jacobian is its general form; slope and gradient are its $m = n = 1$ and $m = 1$ special cases.

Chain rule: composition becomes multiplication

L7′s chain rule multiplied slopes. Jacobians **are** the local linear maps — and composing linear maps is matrix multiplication (L4):

$$J_{g\circ f}(\boldsymbol{a}) = \underbrace{J_g(f(\boldsymbol{a}))}_{k\times m} \cdot \underbrace{J_f(\boldsymbol{a})}_{m\times n} \quad \in \mathbb{R}^{k\times n}$$

Square through f → parallelogram; that parallelogram through g → another parallelogram. Multiply the matrices, compose the leans.

And the shapes **must click**: $(k \times m)(m \times n) = (k \times n)$ — your free error-detector for every derivation.

Planted for L10–L11: a network is $f_L \circ \cdots \circ f_1$, so its derivative is a **product of Jacobians**. Backprop will be nothing but a clever order for multiplying them out.

The derivative table, second visit

V VISUAL

function	derivative	shape	what it is
$f : \mathbb{R} \to \mathbb{R}$	$f'(x)$	a number	slope of the local line (L7)
$f : \mathbb{R}^n \to \mathbb{R}$	$\nabla f(\boldsymbol{x})$	vector, n	all partials; points straight uphill (L8)
$f : \mathbb{R}^n \to \mathbb{R}^m$	$J(\boldsymbol{x})$	matrix, $m \times n$	the local linear map (**today**)
$f : \mathbb{R}^n \to \mathbb{R}$, order 2	**?**	**?**	today

One row left. It needs a different question: not “vector out?” but **“what happened to curvature?”**

Checkpoint: shapes first, always

Q QUESTION

$f(x, y, z) = (x^2 + y^2, y - z)$ maps $\mathbb{R}^3 \to \mathbb{R}^2$. What is the shape of its Jacobian — and is $\det J$ available to talk about areas?

A. 3×2, and yes

B. 2×3, and no

C. 2×3, and yes

D. 3×3, and yes

Answer: shapes first, always

A ANSWER

Correct: B — J is 2×3, and $\det J$ does not exist

Why: One row per output (2), one column per input (3): $J = \begin{pmatrix} 2x & 2y & 0 \\ 0 & 1 & -1 \end{pmatrix}$ is 2×3. Determinants apply only to square matrices; a map $\mathbb{R}^3 \to \mathbb{R}^2$ reduces dimension, so a 3-D volume-scaling factor is not defined.

Curvature becomes a matrix: the Hessian

In one dimension, curvature was one number

L7′s second-derivative test: at a flat point, a **nonzero** f'' settles the local classification — positive gives a minimum, negative a maximum. When $f'' = 0$, higher-order terms decide.

Now stand on a 2-D surface $f : \mathbb{R}^2 \to \mathbb{R}$ and walk in different directions:

- along one street the ground may **curve up** like a smile,
- along the cross-street it may curve up harder, or not at all — **or curve down**.

One number per direction, infinitely many directions. Sounds hopeless — but so did “one slope per direction” before L8 packed every directional slope into **one vector** ∇f.

Same move again, one storey up.

Numbers: the gradient is itself a map we know

Take $f(x, y) = x^2 + xy + y^2$ (keep it in view — it returns all lecture). Its gradient:

$$\nabla f(x, y) = \begin{pmatrix} 2x + y \\ x + 2y \end{pmatrix}$$

Look at what that **is**: vector in, vector out — $\nabla f : \mathbb{R}^2 \to \mathbb{R}^2$. We built the perfect tool for those **ten minutes ago**.

Take the Jacobian **of the gradient**:

$$J_{\nabla f} = \begin{pmatrix} \frac{\partial}{\partial x}(2x + y) & \frac{\partial}{\partial y}(2x + y) \\ \frac{\partial}{\partial x}(x + 2y) & \frac{\partial}{\partial y}(x + 2y) \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

That matrix has a name — and for this f, it is **L5's running matrix** $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. The quadratic-form section makes the connection exact.

The Hessian, defined

$$H(\boldsymbol{x}) = J_{\nabla f}(\boldsymbol{x}), \qquad H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} = \text{how slope } i \text{ responds to a nudge of } x_j$$

- Diagonal H_{ii}: L7′s old curvature along axis i — walk east, does the eastward slope grow?
- Off-diagonal H_{ij}: the **twist** — walk north, does the **eastward** slope change?

For any twice-continuously-differentiable f, mixed partials agree: $H_{ij} = H_{ji}$ (Schwarz’s theorem).

If the second partial derivatives are continuous, the Hessian is symmetric (Schwarz’s theorem). The spectral theorem then gives real eigenvalues and an orthonormal eigenbasis.

The curvature along one direction

Walking from $\boldsymbol{a}$ in unit direction $\boldsymbol{u}$, the ground under you is the 1-D function $g(t) = f(\boldsymbol{a} + t\boldsymbol{u})$, and (chain rule, twice):

$g''(0) = \boldsymbol{u}^\top H \boldsymbol{u}$

For our f with $H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, feel three streets:

direction u	$u^\top H u$	**the street feels like**
$(1, 0)$	2	a smile
$(1, 1)/\sqrt{2}$	$(2 + 1 + 1 + 2)/2 = 3$	the **steepest** smile
$(1, -1)/\sqrt{2}$	$(2 - 1 - 1 + 2)/2 = 1$	the gentlest smile

3 and 1 are H's **eigenvalues**, felt along its **eigenvectors** (L5: $\lambda = 3, 1$ at $(1, 1), (1, -1)$). The extreme curvatures of a surface are an eigen-problem.

Same picture, more rows: a 3-variable Hessian

$f(x, y, z) = x^2 + y^2 + xyz$. First the gradient, then differentiate **it**, coordinate by coordinate:

$$\nabla f = \begin{pmatrix} 2x + yz \\ 2y + xz \\ xy \end{pmatrix} \quad \Rightarrow \quad H = \begin{pmatrix} 2 & z & y \\ z & 2 & x \\ y & x & 0 \end{pmatrix} \quad \underset{(2,1,3)}{\Rightarrow} \quad \begin{pmatrix} 2 & 3 & 1 \\ 3 & 2 & 2 \\ 1 & 2 & 0 \end{pmatrix}$$

Symmetric — check. Entries depend on where you stand — curvature is **local**, computed per point.

Read $H_{12} = z$ out loud: "how the x-slope responds to nudging y." At $(2, 1, 3)$: nudge y by 0.01 and the x-slope $2x + yz$ climbs by 0.03. Every entry of every Hessian is a sentence like this.

Code: the Hessian in one call

```python
from torch.autograd.functional import hessian

f = lambda v: v[0]**2 + v[0]*v[1] + v[1]**2
hessian(f, torch.tensor([1.0, 1.0]))
# tensor([[2., 1.],
#         [1., 2.]])

g = lambda v: v[0]**2 + v[1]**2 + v[0]*v[1]*v[2]
hessian(g, torch.tensor([2.0, 1.0, 3.0]))
# tensor([[2., 3., 1.], [3., 2., 2.], [1., 2., 0.]])
```

Under the hood it does what we did: differentiate the gradient — the Jacobian of ∇f.

The shape gallery: quadratic forms

The pure-curvature family

Which functions have the **same** Hessian everywhere — curvature standing still so we can stare at it? Quadratics. For a symmetric A, multiplied out once by hand on 2×2:

$$q(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top A\boldsymbol{x} = \frac{1}{2}(x_1, x_2)\begin{pmatrix} a & b \\ b & c \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \frac{1}{2}(ax_1^2 + 2bx_1x_2 + cx_2^2) \quad \text{— a number}$$

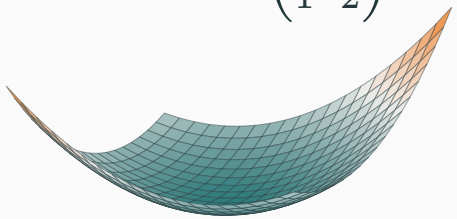
For $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: $q = x^2 + xy + y^2$ — **our running f**. Claim (⭐ proof in the next section): $\nabla q = A\boldsymbol{x}$ and $H = A$, at every point.

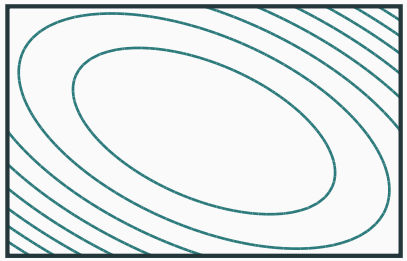
A quadratic form is a Hessian standing still. Classify its shapes, and you've classified all local curvature.

The gallery: bowls, saddles, troughs

V VISUAL

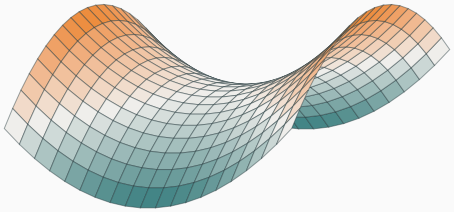
bowl $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$

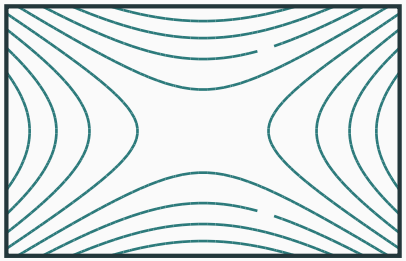




$\lambda = 3, 1$ – both > 0

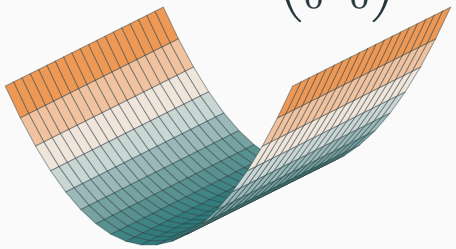
saddle $A = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$

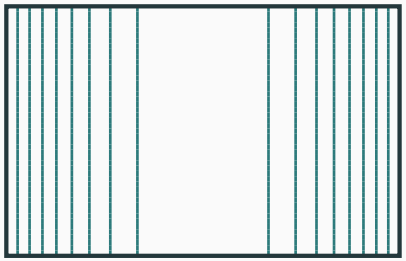




$\lambda = 2, -2$ – mixed signs

trough $A = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$





$\lambda = 2, 0$ – a zero

surfaces and their contour maps, computed live from $q(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top A\boldsymbol{x}$; eigenvalues by the in-slide solver — ellipses, hyperbolas, parallel lines

Eigenvalues read the quadratic shape

A is symmetric, so L5 fires: $A = Q\Lambda Q^\top$ — "a perpendicular grid of dials". In eigen-coordinates $t = Q^\top x$ the cross-term dies; the dials are **curvatures**:

$$q = \frac{1}{2}(\lambda_1 t_1^2 + \lambda_2 t_2^2 + \cdots + \lambda_n t_n^2) \quad \text{— one independent parabola per eigen-direction}$$

eigenvalue signs	quadratic shape	second-order model at a critical point
all $\lambda_i > 0$	bowl	strict local minimum — every street smiles
all $\lambda_i \geq 0$, some $= 0$	trough	flat in zero-curvature directions; higher-order terms decide the actual point
mixed signs	saddle	up some streets, down others — not a min!
all $\lambda_i < 0$	dome	local maximum

Why a zero Hessian eigenvalue is inconclusive

Along a zero-curvature direction, the quadratic term says nothing. Higher orders can differ:

- $f(x, y) = x^2 + y^4$ still has a minimum at the origin,
- $f(x, y) = x^2 - y^4$ has a saddle there,
- both have Hessian $\begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$ at the origin.

Positive definite and indefinite Hessians classify a critical point; a semidefinite Hessian with zeros requires more analysis.

The official words: definiteness

The vocabulary every ML paper uses for "which shape is it":

A is...	definition: for all $x \neq 0$	equivalently
positive definite (PD)	$x^\top Ax > 0$	all $\lambda_i > 0$
positive semi-definite (PSD)	$x^\top Ax \geq 0$	all $\lambda_i \geq 0$
negative definite (ND)	$x^\top Ax < 0$	all $\lambda_i < 0$
indefinite	both signs occur	mixed-sign λ_i

In eigen-coordinates $x^\top Ax = \sum_i \lambda_i t_i^2$; the form is nonnegative for every x exactly when every $\lambda_i \geq 0$.

Positive definite = the surface is a bowl. This is the reading used throughout optimization (L16–L18) and covariance (L13).

Hand tests for 2×2 — ten-second shape reading

L5′s sanity checks were $\operatorname{tr} A = \sum \lambda_i$ and $\det A = \prod \lambda_i$. For symmetric 2×2 they **decide the shape**:

test	**classification**
$\det A < 0$	λ's have opposite signs — **saddle**, done (a negative det always betrays a flip: L4, L5)
$\det A > 0$	same signs — read $\operatorname{tr} A$: positive $\to$ **bowl**, negative $\to$ **dome**
$\det A = 0$	some $\lambda = 0$ — **semi-definite**: trough (or flat)

For $n \times n$, the computer's test is eigenvalues; the classical hand test is **Sylvester's criterion** (all leading principal minors $> 0 \Leftrightarrow$ PD — stated, not proved).

Positive **entries** do not make a matrix positive **definite**: $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ is all-positive yet $\det = -3 < 0$ — a saddle. Definiteness lives in the eigenvalues, not the ink.

Numbers: classify three matrices

A	tr	det	classification by hand	λ (live)
$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$	4	3	det > 0, tr > 0 — **bowl** (PD)	$3, 1$
$\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$	2	-3	det < 0 — **saddle** (indefinite)	$3, -1$
$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$	2	0	det $= 0$, tr > 0 — **trough** (PSD)	$2, 0$

The λ column is computed by the in-slide eigensolver at compile time — trace and det agree with $\sum \lambda_i$ and $\prod \lambda_i$ on every row (L5′s two sanity checks, still working).

Code: three lines settle any matrix

```python
import numpy as np

for A in ([[2, 1], [1, 2]], [[1, 2], [2, 1]], [[1, 1], [1, 1]]):
    lam = np.linalg.eigvalsh(A)      # symmetric -> real, sorted
    print(lam)
# [1. 3.]     all > 0        -> bowl    (PD)
# [-1. 3.]    mixed signs    -> saddle  (indefinite)
# [0. 2.]     a zero, rest + -> trough  (PSD)
```

`eigvalsh` — the `h` means **Hermitian/symmetric**: it promises real eigenvalues because L5′s spectral theorem promises them first.

Checkpoint: read the shape

Q QUESTION

At a critical point ($\nabla f = 0$), the Hessian is $H = \begin{pmatrix} 4 & -2 \\ -2 & 1 \end{pmatrix}$. Its second-order Taylor model is a…

A. bowl — strict local minimum

B. saddle

C. trough — a flat direction

D. dome — local maximum

Answer: read the shape

A ANSWER

Correct: C — trough — a flat direction in the quadratic model

Why: $\det H = 4 - 4 = 0$, so one λ is exactly 0; $\operatorname{tr} H = 5 > 0$ pins the other at $+5$. The quadratic model has positive curvature along one eigen-direction and zero curvature along $(1, 2)$. The Hessian test is therefore inconclusive about the **actual** critical point: higher-order terms along the flat direction can produce a minimum, maximum, or saddle.

The one derivation: the gradient of a quadratic form

Why this gradient, of all gradients

$x^\top Ax$ is the most-differentiated expression in machine learning:

- least squares (L4′s preview): $\|X\theta - y\|^2$ expands into one;
- the Gaussian's exponent (L13) **is** one; the L2 regularizer $\theta^\top\theta$ is one with $A = I$;
- every second-order Taylor model (ten minutes from now) ends in one.

So we earn its gradient honestly — today's ⭐:

Plan: brute-force the 2×2 case symbol by symbol (no matrix calculus allowed), **then** watch the general answer repack itself. One derivation, everything else today is plausibility.

⭐ Step 1: multiply the 2×2 out, no shortcuts

D DERIVATION

Let $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ (deliberately **not** symmetric) and $\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$:

$$\boldsymbol{x}^\top A \boldsymbol{x} = (x_1, x_2) \begin{pmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{pmatrix}$$

$$= ax_1^2 + bx_1x_2 + cx_1x_2 + dx_2^2 = ax_1^2 + (b+c)x_1x_2 + dx_2^2$$

A plain two-variable polynomial — L8 can differentiate this in its sleep. Note who survived: b and c only ever appear as the **sum** $b + c$.

⭐ Step 2: two honest partial derivatives

D DERIVATION

$$\frac{\partial}{\partial x_1}\left[ax_1^2 + (b+c)x_1x_2 + dx_2^2\right] = 2ax_1 + (b+c)x_2$$

$$\frac{\partial}{\partial x_2}\left[ax_1^2 + (b+c)x_1x_2 + dx_2^2\right] = (b+c)x_1 + 2dx_2$$

Stack them into the gradient (our column convention):

$$\nabla(\boldsymbol{x}^\top A\boldsymbol{x}) = \begin{pmatrix} 2ax_1 + (b+c)x_2 \\ (b+c)x_1 + 2dx_2 \end{pmatrix}$$

⭐ Step 3: recognize the matrix hiding in it

D DERIVATION

$$\begin{pmatrix} 2ax_1 + (b+c)x_2 \\ (b+c)x_1 + 2dx_2 \end{pmatrix} = \begin{pmatrix} 2a & b+c \\ b+c & 2d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \left[\begin{pmatrix} a & b \\ c & d \end{pmatrix} + \begin{pmatrix} a & c \\ b & d \end{pmatrix} \right] \boldsymbol{x}$$

$$\nabla(\boldsymbol{x}^\top A\boldsymbol{x}) = (A + A^\top)\boldsymbol{x} \qquad \Rightarrow \quad \text{symmetric } A: \quad \nabla = 2A\boldsymbol{x}$$

Same argument in $\mathbb{R}^n$, one line: $\boldsymbol{x}^\top A\boldsymbol{x} = \sum_{i,j} A_{ij}x_i x_j$, and $\partial/\partial x_k$ harvests row k ($\sum_j A_{kj}x_j$) plus column k ($\sum_i A_{ik}x_i$) — that is $(A\boldsymbol{x})_k + (A^\top\boldsymbol{x})_k$. Same picture, more rows.

Corollary (one more derivative): $H = J_\nabla = A + A^\top$ — for symmetric A and $q = \frac{1}{2}\boldsymbol{x}^\top A\boldsymbol{x}$: $\nabla q = A\boldsymbol{x}$, $H = A$. The gallery's caption is now a theorem.

Numerical check of the rule

The slide checks itself. Chalkdust's autodiff evaluates $\nabla(\boldsymbol{x}^\top A\boldsymbol{x})$ for $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ at $(1, 1)$ — reverse mode, no symbols:

$$\text{autodiff: } \nabla = \begin{pmatrix} 6.0000 \\ 6.0000 \end{pmatrix} \qquad \text{our rule: } 2A\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 6.0000 \\ 6.0000 \end{pmatrix} \quad \text{— identical } \checkmark$$

Two rules you now own for free, by choosing A:

expression	gradient	why
$\|\boldsymbol{x}\|^2 = \boldsymbol{x}^\top I\boldsymbol{x}$	$2\boldsymbol{x}$	$A = I$ is symmetric
$\boldsymbol{b}^\top\boldsymbol{x}$ (linear)	$\boldsymbol{b}$	no square term — the slope is constant

These two + today's ⭐ generate most of the Matrix Cookbook's first page. The rest is looked up, not re-derived — more on that in the bookkeeping section.

Taylor, one degree at a time

L7's ladder, one more time

L7 replaced a function near x_0 by polynomials, one degree at a time:

$$f(x_0 + h) \approx f(x_0) + f'(x_0)\,h + \frac{1}{2} f''(x_0)\,h^2$$

Every ingredient just grew up. Upgrade each term with today's table:

1-D ingredient	**n-D upgrade**	**the term becomes**
step h	step vector $\boldsymbol{\delta}$	
$f'(x_0)\,h$	∇f (L8)	$\nabla f(\boldsymbol{x})^\top \boldsymbol{\delta}$ — a dot product
$\frac{1}{2} f''(x_0)\,h^2$	H (today)	$\frac{1}{2} \boldsymbol{\delta}^\top H(\boldsymbol{x}) \boldsymbol{\delta}$ — a quadratic form

Slope became a vector, curvature became a matrix — and h^2 became "sandwich H between two $\boldsymbol{\delta}$'s".

The second-order local model

$$f(\boldsymbol{x}+\boldsymbol{\delta}) \approx \underbrace{f(\boldsymbol{x})}_{\text{height}} + \underbrace{\nabla f(\boldsymbol{x})^\top \boldsymbol{\delta}}_{\text{tilt: the local plane}} + \underbrace{\frac{1}{2}\boldsymbol{\delta}^\top H(\boldsymbol{x})\,\boldsymbol{\delta}}_{\text{bend: the local bowl or saddle}}$$

Read it term by term: near $\boldsymbol{x}$, the approximation is a **height, plus a tilt, plus a quadratic curvature term**. Eigenvalue signs classify that quadratic term; zero eigenvalues leave some directions to higher-order analysis.

Later, gradient descent uses the linear term to choose a descent direction, while Newton's method minimizes the quadratic model.

Numbers: predict a function you've never met

$f(x,y) = x^2 y$ at $\boldsymbol{a} = (1,1)$, step $\boldsymbol{\delta} = (0.1, 0.1)$. Harvest the three ingredients:

$$f(\boldsymbol{a}) = 1, \qquad \nabla f = \begin{pmatrix} 2xy \\ x^2 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad H = \begin{pmatrix} 2y & 2x \\ 2x & 0 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 2 & 0 \end{pmatrix}$$

Assemble, term by term:

$$f(\boldsymbol{a}+\boldsymbol{\delta}) \approx 1 + \underbrace{(2,1)\cdot(0.1,0.1)}_{0.3} + \underbrace{\frac{1}{2}(0.1,0.1)\begin{pmatrix} 2 & 2 \\ 2 & 0 \end{pmatrix}\begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}}_{\frac{1}{2}(0.06)=0.03} = 1.33$$

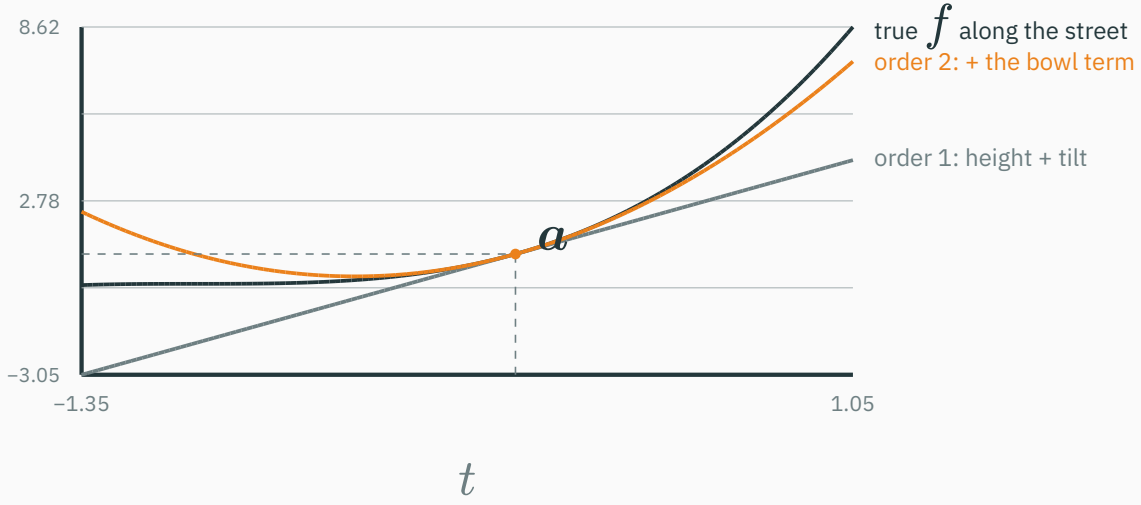
Truth: $f(1.1, 1.1) = 1.1^2 \times 1.1 = 1.331$. Off by 0.001 — exactly the dropped third-order term.

Note the Hessian is **indefinite** here ($\det = -4 < 0$: saddle curvature) — Taylor doesn't care; it reports the local shape, whatever it is.

Three approximations along one line

V VISUAL

Walk from $\boldsymbol{a} = (1, 1)$ along $\boldsymbol{\delta} = (t, t)$: the true f becomes $g(t) = (1 + t)^3$, and the Taylor terms become L7-style line and parabola — all three plotted live:



Near $\boldsymbol{a}$, the quadratic approximation follows the truth visibly longer than the linear one — the multivariate formula reduced to L7′s one-dimensional picture.

Code: the whole expansion in five lines

```
from torch.autograd.functional import jacobian, hessian

f = lambda v: v[0]**2 * v[1]
a = torch.tensor([1.0, 1.0]); d = torch.tensor([0.1, 0.1])

g, H = jacobian(f, a), hessian(f, a)
pred2 = f(a) + g @ d + 0.5 * d @ H @ d
# pred2 = 1.3300     true f(a + d) = 1.3310    gap = 3rd order
```

Five lines you will reuse in T5; the same local model appears in the second-order optimizers of Module 4.

Where this formula is used later

lecture	what it does with today's expansion
L16	**convexity**: H PSD everywhere — the surface is one global bowl, no bad valleys
L17	**gradient descent**: use the local linear term for a finite step; H's eigenvalues determine the stable learning-rate range
L18	**Newton's method**: trust the bowl — minimize the quadratic model, jump $\delta^* = -H^{-1}\nabla f$

Many optimization methods can be compared by which local information they use and how they control the step size.

Bookkeeping & the take-home

Our contract: the gradient is a column

Matrix calculus uses two common bookkeeping conventions. **Numerator layout** writes the derivative of a scalar as a row; **denominator layout** as a column. Both are consistent; mixing them mid-derivation creates transposition errors.

This course signs one contract and sticks to it (MML's):

object	**shape**	**note**
$\boldsymbol{x}$, $\boldsymbol{\delta}$, ∇f	column, n	**gradient-as-column** — same shape as $\boldsymbol{x}$, so $\boldsymbol{x} - \eta\nabla f$ type-checks (L17)
J of $f : \mathbb{R}^n \to \mathbb{R}^m$	$m \times n$	rows = outputs; for $m = 1$, $J = \nabla f^\top$ (a row)
H	$n \times n$	symmetric; $H = J_{\nabla f}$

When you open any other book or blog, **find its convention first** — a transposed-looking formula is usually the other school, not an error.

Shape-checking is a proof technique

The chain rule **forces** every shape, so let the shapes catch your bugs:

$$\underbrace{J_{g\circ f}}_{k\times n} = \underbrace{J_g}_{k\times m}\underbrace{J_f}_{m\times n}, \qquad \underbrace{\nabla f(\boldsymbol{x})^\top \boldsymbol{\delta}}_{(1\times n)(n\times 1)=\text{ scalar}}, \qquad \underbrace{\boldsymbol{\delta}^\top}_{1\times n}\underbrace{H}_{n\times n}\underbrace{\boldsymbol{\delta}}_{n\times 1} = \text{scalar}$$

If a derivation produces a shape mismatch, it is wrong — no further reading required. That five-second check replaces half of matrix-calculus memorization.

References: the Matrix Cookbook for identities; Parr & Howard, *The Matrix Calculus You Need for Deep Learning* for an accessible derivation; MML §5.4–5.7 for the course convention.

Apply both derivatives to the opening model

A neural network is a chain $\mathcal{L} = f_L \circ \cdots \circ f_1(\theta)$. What is its derivative? Now we can **name every piece**:

1. each layer $f_i : \mathbb{R}^n \to \mathbb{R}^m$ has a **Jacobian** — a lean-and-stretch parallelogram map;
2. the chain rule multiplies them: $J = J_L \cdots J_1$;
3. the last output is the scalar loss, so the whole product collapses to **one row** — the gradient $\nabla \mathcal{L}^\top$, L8′s uphill vector, for a million inputs;

The Jacobian is the local linear map for vector outputs. L10–L11 compute and compose these derivatives efficiently.

Why a full Hessian is rarely stored

The scalar loss also has a curvature matrix, the **Hessian**. At $d = 10^6$ parameters it contains 10^{12} entries, about 4 TB in float32.

Large-model optimizers therefore use approximations or Hessian-vector products rather than store the matrix explicitly. L17–L18 develop those uses.

The derivative table, complete

V VISUAL

function	derivative	shape	what it is
$f : \mathbb{R} \to \mathbb{R}$	$f'(x)$	a number	slope of the local line (L7)
$f : \mathbb{R}^n \to \mathbb{R}$	$\nabla f(\boldsymbol{x})$	vector, n	all partials; points straight uphill (L8)
$f : \mathbb{R}^n \to \mathbb{R}^m$	$J(\boldsymbol{x})$	matrix, $m \times n$	the local linear map (**today**)
$f : \mathbb{R}^n \to \mathbb{R}$, order 2	$H(\boldsymbol{x})$	matrix, $n \times n$, symmetric	curvature, in every direction (**today**)

One idea, four containers: derivatives parameterize local linear approximations, and the Hessian supplies the quadratic term for a scalar output. The array shape follows the function's input and output shapes.

Lecture 9 — summary

- **Jacobian**: $m \times n$ array of partial derivatives; the local linear map $\Phi(\boldsymbol{a} + \boldsymbol{\delta}) \approx \Phi(\boldsymbol{a}) + J\boldsymbol{\delta}$; $|\det J|$ gives local area scaling for square maps (polar: $\det J = r$).
- **Chain rule** = matrix multiplication of Jacobians; shapes must click ($(k \times m)(m \times n)$).
- **Hessian** = Jacobian of the gradient; symmetric when second partials are continuous; entry (i, j) = "how slope i feels nudge j"; $\boldsymbol{u}^\top H \boldsymbol{u}$ = curvature along $\boldsymbol{u}$, extremes at eigenvectors.
- **Quadratic forms** $\frac{1}{2}\boldsymbol{x}^\top A\boldsymbol{x}$: bowls (all $\lambda > 0$), saddles (mixed), troughs (a zero λ); 2×2 hand test via det and trace; ⭐ $\nabla(\boldsymbol{x}^\top A\boldsymbol{x}) = (A + A^\top)\boldsymbol{x}$.
- **Taylor, order 2**: $f + \nabla f^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top H \boldsymbol{\delta}$ — height, tilt, bend; the local model L17–L18 optimize.

Before L10

- Read MML §5.4–5.7 and skim Parr & Howard's matrix-calculus guide.
- Keep the Matrix Cookbook as a lookup reference; do not try to memorize it.
- T5, after L11, practises Jacobians and a complete backward pass.

Practice problems

Try on paper; verify with `torch.autograd.functional` as in today's code slides.

1. $F(x, y) = (x^2 y, x + y^3)$: compute J at $(1, 1)$ and $\det J$. Is F locally inflating or crushing areas there — by what factor?
2. For $\Phi(x, y) = (e^x \cos y, e^x \sin y)$: show $\det J = e^{2x}$. Why can this map never crush area to zero?
3. $f(x, y) = x^3 + y^3 - 3xy$: find both critical points, compute H at each, and classify (one saddle, one bowl).
4. $f(x, y) = xy$ has **no** squared term anywhere. Compute H, its eigenvalues, and the shape at $(0, 0)$.
5. Apply the ⭐ rule to the non-symmetric $A = \begin{pmatrix} 1 & 4 \\ 0 & 1 \end{pmatrix}$: write $\nabla(\boldsymbol{x}^\top A \boldsymbol{x})$, then verify by expanding $\boldsymbol{x}^\top A \boldsymbol{x}$ by hand.
6. Second-order Taylor of $f(x, y) = e^x \sin y$ at $(0, \frac{\pi}{2})$: show $\nabla f = (1, 0)$, $H = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, predict $f(0.1, \frac{\pi}{2} + 0.1)$, and compare with the true value 1.0996.

⭐⭐⭐ Beyond order two: derivatives become tensors

★ OPTIONAL

Why does everyone stop at the Hessian? Keep differentiating and count indices:

order	object	entries at $d = 10^6$
1	∇f — vector	10^6 (4 MB)
2	H — matrix	10^{12} (4 TB)
3	$\partial^3 f$ — a 3-index **tensor**	10^{18} (4 EB — exabytes)

Storage explodes by d per order, so large-scale ML usually uses first derivatives, sometimes exploits second-order information, and seldom materializes third-order derivative tensors.

Escape hatch, teased for L11 ⭐⭐⭐: you can compute **Hessian-vector products** Hv without ever materializing H — differentiate $\nabla f^\top v$ once more. Curvature information at gradient prices.

Curvature is a matrix, and its sign is the shape of the bowl.

Next: **Differentiation on a Computer** — the computer takes over differentiation: finite differences fight floating point (L2 returns), and forward mode wins.